Parallel Performance Analysis of Ant Colony Optimization with OpenMP, MPI, and Hybrid Domain Decomposition

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

2nd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

3rd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

4th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

5th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

6th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

***Abstract*—This document analyzes the parallel execution of an Ant Colony Optimization simulation over a fractal terrain. The study compares a shared-memory OpenMP implementation, a distributed-memory MPI implementation based on subdomain decomposition with halo exchange and ant migration, and a hybrid MPI+OpenMP variant that introduces intra-subdomain thread-level parallelism. The reported results focus on iteration time, computation time, rendering cost, and communication overhead across multiple colony sizes in order to identify the practical scalability limits of each model and the tradeoffs between computation and communication.**

***Index Terms*—ant colony optimization, parallel computing, OpenMP, MPI, hybrid programming, domain decomposition**

I. Introduction

The Ant Colony Optimization problem is one of the most widely studied metaheuristics for the simulation of collective behaviors in nature. The implementation of the algorithm is proposed from the definition of multiple artificial ants, either as a class or from the vector-based description of their properties. Each of these artificial entities makes movement decisions based on a combination of local heuristic information and a shared collective memory, usually modeled through an artificial pheromone. This optimization approach was introduced by Marco Dorigo in the 1990s [1].

The importance of this optimization lies in its ability to address problems in which the search space grows rapidly and where exact methods face difficulties in terms of computational scalability. Nevertheless, in large-scale scenarios, not only because of the size of the ant colony but also due to the complexity of the map, limitations may arise in computation times and in the efficient handling of increasingly larger scales.

In this context, the imminent need emerges to incorporate scalability and parallelization strategies for ACO, aimed at reducing the execution time of each algorithm by decreasing the computational load and facilitating the treatment of even larger entities. For this reason, the present document addresses parallelization with shared memory through OpenMP, as well as two different distributed-memory parallelization strategies using MPI, with the objective of improving the handling of large instances in the description of the collective behavior of ants.

A. Simulation times measurement

The measured times are defined so as to separate the simulation into clearly differentiated phases, which makes it possible to identify the contribution of each operation to the overall computational cost. In `event_poll` (see Table I), the time spent handling SDL events is recorded, including window closing, basic interaction, and event-queue updates; this corresponds to interface overhead rather than algorithmic work.

In `ants_advance` (Table I), the behavioral core of the ants is measured, including movement decisions, neighboring pheromone reads, food transport, collection accounting, and pheromone marking. In `evaporation` (Table I), only the pheromone evaporation stage is measured, which is characterized by ordered memory traversal and conditional evaporation logic and is therefore favorable for locality- and vectorization-based optimization. In `update` (Table I), the consolidation of the pheromone state at the end of the step is included, together with buffer exchanges, restoration of special cells, reimposition of nest and food sources, and map maintenance. Finally, `advance_total` (Table I) provides the most representative measure of the simulation core cost because it integrates `ants_advance`, `evaporation`, and `update`.

In `render` (Table I), the time required to draw the scene is measured, including the `blit` stage and the full screen-frame

representation cost. All these contributions are ultimately included in `iteration_total` (Table I), which represents the complete cost of one simulation iteration.

Additionally, general simulation information is stored, including the number of measured iterations, the final amount of food after the simulation ends, and the iteration at which the first food unit arrives.

TABLE I
SUMMARY OF MEASURED EXECUTION TIMES IN THE SIMULATION.

Time metric	Brief description
`event_poll`	SDL event handling.
`ants_advance`	Ant movement and decisions.
`evaporation`	Pheromone evaporation.
`update`	State consolidation and restoration.
`advance_total`	Total simulation-core cost.
`render`	Frame rendering time.
`iteration_total`	Total iteration time.

II. DISTRIBUTED SUBDOMAIN MPI

In this section, the implementation of an optimization strategy over the fractal domain is proposed based on the decomposition of the terrain into two-dimensional subdomains, which are managed by each MPI process. Each one is responsible for managing one of these subdomains in the map, considering the boundary effects between two-dimensional subdomains through the concept of ghost cells. Additionally, only the logic for generating a global terrain is preserved in the root node for data collection and subsequent rendering.

The domain decomposition is constructed through a 2D MPI Cartesian topology. For this, `MPI_Dims_create` is used for the automatic factorization of the processes into a two-dimensional mesh of dimensions. Then, Cartesian communicators are created for the direct neighbors in the four directions: up, down, left, and right. In this way, each of the processes knows its Cartesian coordinates associated with the fractal terrain, as well as its local size and offset in both directions.

In addition to the interior block, each of the processes preserves a ghost-cell crown of thickness determined by parameterization in the case of the simulation, which is taken into account for local storage in the stride variables. The memory linear accesses as well as the indexations are performed considering the effect of those ghost cells that are implemented for the management of the neighborhood of the subdomains and the boundary conditions. It is this ghost-cell structure that makes it possible for MPI communication not to be required for each one of the ants that is located at the edge or boundary of its subdomain, considering that the algorithm only needs the immediate neighbors at a single-cell distance; therefore, a single layer of thickness 1 is sufficient to satisfy the algorithmic requirement.

The exchange of ghost cells for the fields that are static during the simulation, as in the case of the terrain and the cell type, is carried out with a classical one-halo-layer-per-boundary policy. Each process packs its first and last interior row for up/down, and its first and last interior column for left/right; then it launches non-blocking `MPI_Irecv` and `MPI_Isend` with the corresponding neighbors.

A. Pheromone system

The pheromone system is organized into two channels, each of which has its own memory buffer, which allows the distinction of two trails with a different semantics: an ant without load consults channel 0, while a loaded ant consults channel 1. To avoid coherence problems between the current pheromone state and the next one, two copies are preserved, one current and one next, so that all ants read the current version but write to the next version, and at the end of the step the exchange or swap is performed.

In terms of the exchange, instead of sending `v1` and `v2` separately between the subdomains, what is done is to pack them into a single interleaved buffer, for each one of the cells at the boundary of the subdomains. This provides a reduction in the number of messages to be sent with respect to independent fields, mainly because the communication costs exceed the cache-query costs associated with the buffer.

B. Ant migration

When an ant, as part of its movement, crosses a boundary and therefore the destination cell no longer belongs to the process to which this ant is assigned, a `TransitAnt` is constructed that contains the information associated with the properties of the ant, including the information for the pheromone update in the cell into which the ant enters before migration. The migration protocol is implemented with two neighborhood collectives over the Cartesian communicator. First, each process exchanges with its four neighbors the counts of outgoing ants per direction (up/down/left/right) by using `MPI_Neighbor_alltoall`. Then, with the received counts, it exchanges the payload arrays through `MPI_Neighbor_alltoallv` with `MPI_BYTE`, transmitting contiguous blocks of `TransitAnt` with size `count * sizeof(TransitAnt)`.

Finally, it concatenates what is received into an `active` vector. The ants that arrive at the process do not mean that they have already completed their step; that is why it calls `process_single_ant(...)` on each received active ant. That function continues the movement from the partial state of the ant; and if during that continuation it crosses another boundary again, it sends it again to another neighbor. The loop is not exited until the entire system remains "at rest" with respect to migrations, that is, until there is no ant pending to complete its distributed step. The global active-ant total is computed with `MPI_Iallreduce(..., MPI_SUM, ...)` and checked periodically (not every migration round), reducing global synchronization frequency while preserving the same completion criterion (`global_active == 0`).

C. Time measurement

Given the particular conditions of the fractal-terrain partitioning and the ownership of ants over two-dimensional subdomains, it is necessary to add new timing measurements

in order to identify the computational costs of the strategies implemented for parallelization with distributed memory. The time `mpi_halo_exchange` is included to measure the time taken by the halo exchange of pheromones in each iteration, which accounts for the communication time associated with the fixed neighborhood imposed by the spatial decomposition.

`mpi_migration` measures the time spent within `advance` in the exchange of ants between one rank and another, and is therefore responsible for measuring the cost associated with the distribution of the ant population across the subdomains handled by the processes. For its part, the variable `mpi_food_allreduce` accounts for the blocking wait time associated with the non-blocking global food reduction (`MPI_Iallreduce + MPI_Wait`), after overlapping part of that communication with local computation.

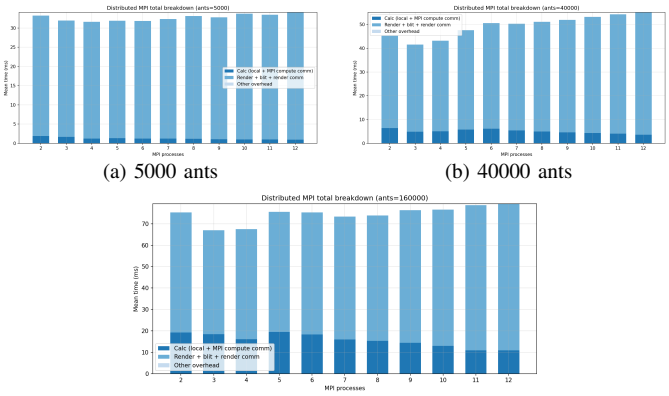
`mpi_render_comm` measures `gather_state_for_render(...)`, that is, the collection of the distributed state toward rank 0 in order to render pheromones and ants in a single global view. It is not the rendering itself, but rather the previous communication required for rendering. Finally, `halo_ms_local + migration_ms_local + food_allreduce_ms_local + render_comm_ms_local` are accumulated into `mpi_total_comm`, which accounts for the fraction of the total time spent in communication between MPI processes. Additionally, to reduce synchronization overhead in reporting, per-iteration timing reductions are aggregated and reduced in batches instead of reducing every metric in every iteration.

TABLE II
ADDITIONAL MPI COMMUNICATION TIMING METRICS.

Time metric	Brief description
`mpi_halo_exchange`	Pheromone halo exchange.
`mpi_migration`	Ant migration between ranks.
`mpi_food_allreduce`	Global food reduction.
`mpi_render_comm`	State gathering for rendering.
`mpi_total_comm`	Total MPI communication time.

D. Results analysis

It is therefore proposed to analyze the results of the MPI parallelization based on the subdivision into subdomains, and two elements become evident. The first is that the bottleneck in the execution of the simulations, both in the subdomain-based parallelization and in the other parallelization strategies that have been analyzed, lies in the graphical part of the algorithm as shown in Fig. 1. Even in the case in which the number of ants, and therefore the number of computational operations, grows up to 320000 ants in the colony, this causes the effect of the parallelization not to become necessarily evident in the analysis of the total time of each iteration in the simulation. However, if only the time consumed by the algorithm in `ant_advance`, which corresponds to the total computation time of the algorithm, is analyzed, the dynamics differ considerably. In particular, the decomposition by components reveals that the computational stage preserves a favorable tendency with the increase in the number of processes, whereas the visual stage keeps a relatively high weight per iteration. This behavior explains why the global metric can suggest weak scaling even when the numerical core is effectively accelerated by the distributed scheme.



(a) 5000 ants (b) 40000 ants

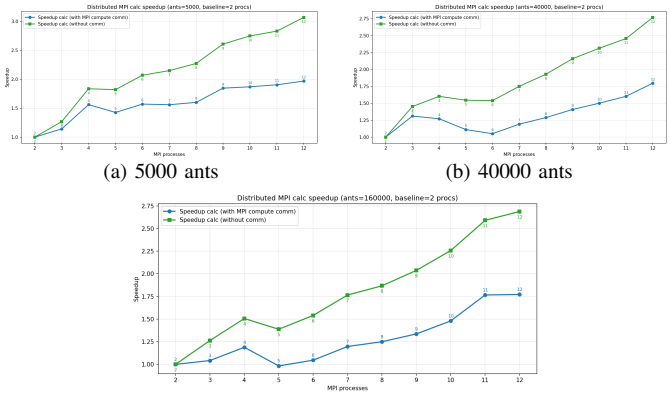
(c) 160000 ants

Fig. 1. MPI subdomain total iteration-time breakdown for selected colony sizes.

When the number of ants is small, as in the case of a colony of 5000 ants, it is possible to observe the effect produced by the incorporation of more processes in the acceleration of the computation, reaching a maximum close to 2 when communications between processes are considered. Nevertheless, when the speedup without communication is considered, a much more marked growth effect can be seen, reaching a speedup greater than 3 for 12 processes. However, this difference between the real speedup of the computation and the speedup without considering communication becomes increasingly larger as more subdomains are added in the representation of the fractal terrain and the communication between processes increases as depicted in Fig. 2 and Fig. 3. The growth in the size of the ant colony leads to similar dynamics in terms of speedup, showing reductions as the number of ants increases and revealing behaviors in which, despite corresponding to a larger number of processes, the linearity of the executed actions or the cost of memory-access operations due to the growth of the buffers causes the resulting effect in terms of speedup to be somewhat smaller than in the case of a colony size of 5000 ants.

Nevertheless, it is evident that, unlike the case of parallelization with shared memory, there is an improvement in performance with the incorporation of processes which, although conditioned by the size of the buffers in which information is stored and also by the cost of communications, means that from the incorporation of more than one computation thread onward, the MPI parallelization shows, over the vast majority of the domain, a better performance per iteration than the OpenMP parallelization, as can be seen in Fig. 4.

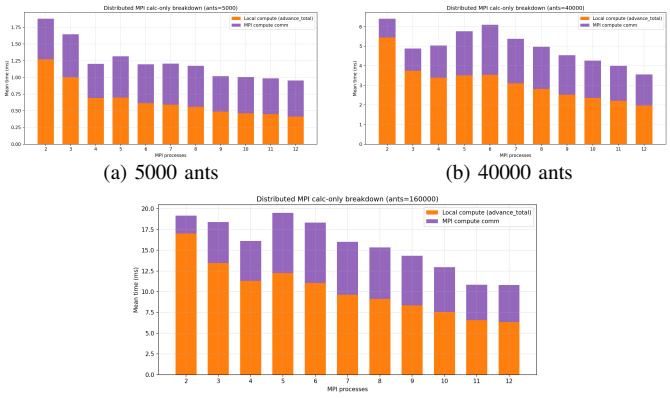
For a direct three-way comparison between OMP-only, MPI-only, and Hybrid MPI+OMP sweeps, the script `docs/scripts/plot_compare_omp_mpi_hybrid.py`



(a) 5000 ants (b) 40000 ants

(c) 160000 ants

Fig. 2. MPI subdomain computation speedup (with and without communication) for selected colony sizes. The associated efficiency is computed with respect to compute MPI processes only, excluding rendering.



(a) 5000 ants (b) 40000 ants

(c) 160000 ants

Fig. 3. MPI subdomain computation-only breakdown for selected colony sizes.

TABLE III
MPI COMPUTE SPEEDUP FOR ALL COLONY SIZES AND MPI PROCESS COUNTS.

Proc/Ants	5k	10k	20k	40k	80k	160k
2	1.00	1.00	1.00	1.00	1.00	1.00
3	1.14	1.33	1.29	1.31	1.22	1.04
4	1.56	1.49	1.21	1.27	1.40	1.19
5	1.43	1.39	1.15	1.11	1.20	0.98
6	1.57	1.44	1.23	1.05	1.14	1.05
7	1.56	1.48	1.42	1.19	1.30	1.20
8	1.60	1.55	1.43	1.29	1.31	1.25
9	1.85	1.65	1.62	1.41	1.38	1.34
10	1.87	1.59	1.66	1.50	1.45	1.48
11	1.90	1.50	1.74	1.60	1.65	1.77
12	1.97	1.79	1.91	1.80	1.73	1.77

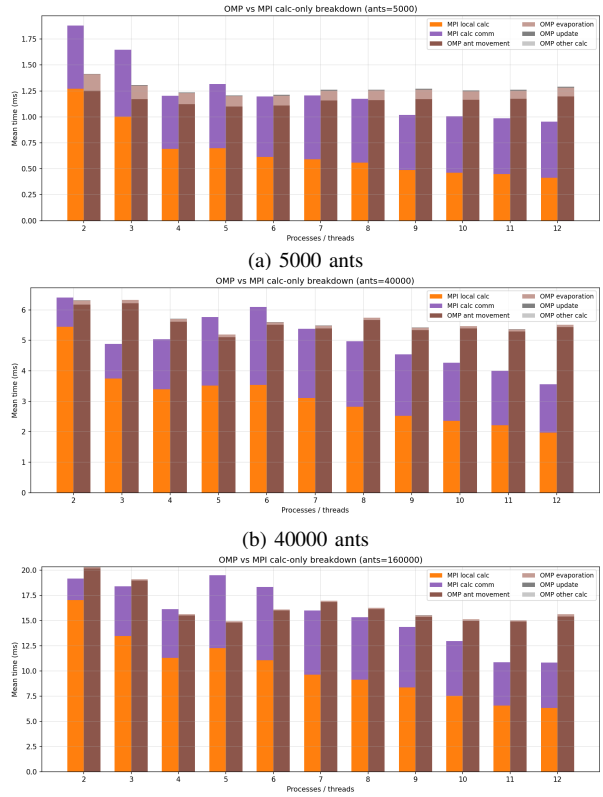
Note. The compute metric is $advance_total + mpi_halo_exchange + mpi_migration + mpi_food_allreduce$, matching the MPI compute plots. Efficiency is computed with respect to the number of compute MPI processes only; rendering is excluded from the process count. For each colony size, the baseline is the smallest available MPI process count p_0. Each cell reports speedup $S = T(p_0)/T(p)$.

TABLE IV
MPI COMPUTE EFFICIENCY FOR ALL COLONY SIZES AND MPI PROCESS COUNTS.

Proc/Ants	5k	10k	20k	40k	80k	160k
2	1.00	1.00	1.00	1.00	1.00	1.00
3	0.76	0.89	0.86	0.87	0.82	0.69
4	0.78	0.75	0.61	0.64	0.70	0.59
5	0.57	0.56	0.46	0.44	0.48	0.39
6	0.52	0.48	0.41	0.35	0.38	0.35
7	0.45	0.42	0.41	0.34	0.37	0.34
8	0.40	0.39	0.36	0.32	0.33	0.31
9	0.41	0.37	0.36	0.31	0.31	0.30
10	0.37	0.32	0.33	0.30	0.29	0.30
11	0.35	0.27	0.32	0.29	0.30	0.32
12	0.33	0.30	0.32	0.30	0.29	0.30

Note. The compute metric is $advance_total + mpi_halo_exchange + mpi_migration + mpi_food_allreduce$, matching the MPI compute plots. Efficiency is computed with respect to the number of compute MPI processes only; rendering is excluded from the process count. For each colony size, the baseline is the smallest available MPI process count p_0. Each cell reports efficiency $E = S/(p/p_0)$.

generates the figures under `docs/figures/compare_omp_mpi_hybrid/`.



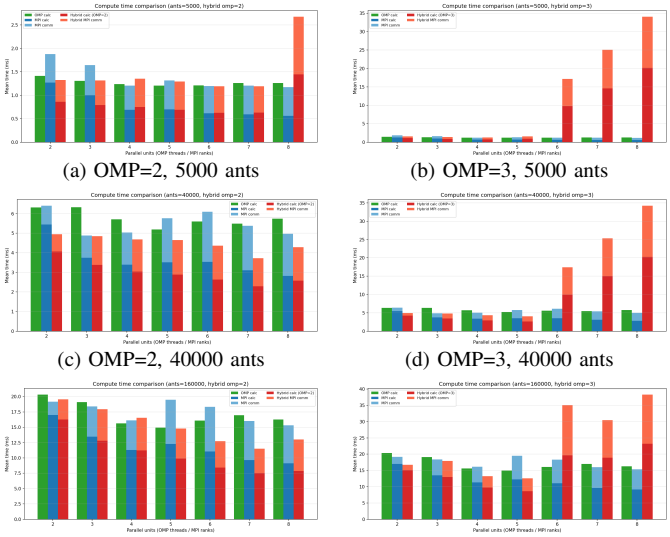
(a) 5000 ants

(b) 40000 ants

(c) 160000 ants

Fig. 4. MPI subdomain vs OpenMP computation-only breakdown for selected colony sizes.

A relevant conclusion is that as has been discused the scaling of the subdomain strategy is jointly limited by com-



(a) OMP=2, 5000 ants (b) OMP=3, 5000 ants

(c) OMP=2, 40000 ants (d) OMP=3, 40000 ants

(e) OMP=2, 160000 ants (f) OMP=3, 160000 ants

Fig. 5. Hybrid OMP-only vs MPI-only vs MPI+OpenMP computation-only comparison for selected colony sizes. The left column corresponds to two OpenMP threads per subdomain and the right column corresponds to three OpenMP threads per subdomain. The associated efficiency is computed with respect to compute MPI processes only, excluding rendering.

munication growth and memory-traffic pressure in migration buffers as the process count and colony size increase. Consequently, although the inclusion of more processes improves performance over a broad operating region, the incremental gain per added process decreases at high process counts, which is consistent with the observed separation between the speedup with communication and the speedup without communication.

E. Hybrid distributed subdomain

Considering the particularities of the solution implemented over the subdomains, the implementation of a hybrid model for the ant colonies is proposed, given that the cost of increasing the number of subdomains using MPI parallelization became evident, but it also became evident that the increase in subdomains causes a considerable increase in the number of communication operations between subdomains. Within this framework, hybrid integration is proposed as a solution so that each one of the subdomains has threads that execute in parallel with shared memory using OpenMP, as contrasted in Fig. 5. The objective is not necessarily to reengineer the algorithm, but rather to integrate shared-memory threads within each subdomain in order to increase parallelism in the computation without incurring the already mentioned costly communication costs associated with an increase in the number of subdomains into which the fractal terrain is decomposed.

From the results of the hybrid implementation, it becomes evident that when two threads are implemented for each process, the computation execution-time results improve according to the number of processes, in most cases even more when the number of ants is larger and the increase in the parallel computing units favors the reduction of computation time. When the loads are medium or large, however, it is still evident that the fragmentation of the domain continues

TABLE V
HYBRID MPI+OPENMP COMPUTE SPEEDUP FOR OMP=2 THREADS PER RANK.

Proc/Ants	5k	10k	20k	40k	80k	160k
2	1.00	1.00	1.00	1.00	1.00	1.00
3	1.01	1.01	1.05	1.02	1.01	1.09
4	0.98	0.97	1.00	1.06	1.06	1.18
5	1.02	1.01	1.02	1.06	1.12	1.32
6	1.11	1.08	1.11	1.14	1.18	1.54
7	1.11	1.16	1.21	1.33	1.35	1.70
8	0.49	0.64	0.96	1.16	1.32	1.51

Note. The compute metric is $advance_total + mpi_halo_exchange + mpi_migration + mpi_food_allreduce$, matching the hybrid sweep plots. Efficiency is computed with respect to the number of compute MPI processes only; rendering is excluded from the process count. For each combination of colony size and OpenMP threads per rank, the baseline is the smallest available MPI process count p_0. Each cell reports speedup $S = T(p_0)/T(p)$.

TABLE VI
HYBRID MPI+OPENMP COMPUTE EFFICIENCY FOR OMP=2 THREADS PER RANK.

Proc/Ants	5k	10k	20k	40k	80k	160k
2	1.00	1.00	1.00	1.00	1.00	1.00
3	0.67	0.67	0.70	0.68	0.67	0.73
4	0.49	0.49	0.50	0.53	0.53	0.59
5	0.41	0.40	0.41	0.43	0.45	0.53
6	0.37	0.36	0.37	0.38	0.39	0.51
7	0.32	0.33	0.35	0.38	0.39	0.49
8	0.12	0.16	0.24	0.29	0.33	0.38

Note. The compute metric is $advance_total + mpi_halo_exchange + mpi_migration + mpi_food_allreduce$, matching the hybrid sweep plots. Efficiency is computed with respect to the number of compute MPI processes only; rendering is excluded from the process count. For each combination of colony size and OpenMP threads per rank, the baseline is the smallest available MPI process count p_0. Each cell reports efficiency $E = S/(p/p_0)$.

TABLE VII
HYBRID MPI+OPENMP COMPUTE SPEEDUP FOR OMP=3 THREADS PER RANK.

Proc/Ants	5k	10k	20k	40k	80k	160k
2	1.00	1.00	1.00	1.00	1.00	1.00
3	1.13	1.14	1.10	1.02	0.96	0.93
4	1.26	1.25	1.19	1.13	1.16	1.26
5	1.00	1.14	1.19	1.21	1.26	1.33
6	0.09	0.13	0.18	0.28	0.44	0.48
7	0.06	0.09	0.12	0.19	0.31	0.55
8	0.05	0.07	0.09	0.14	0.23	0.44

Note. The compute metric is $advance_total + mpi_halo_exchange + mpi_migration + mpi_food_allreduce$, matching the hybrid sweep plots. Efficiency is computed with respect to the number of compute MPI processes only; rendering is excluded from the process count. For each combination of colony size and OpenMP threads per rank, the baseline is the smallest available MPI process count p_0. Each cell reports speedup $S = T(p_0)/T(p)$.

TABLE VIII
HYBRID MPI+OPENMP COMPUTE EFFICIENCY FOR OMP=3 THREADS PER RANK.

Proc/Ants	5k	10k	20k	40k	80k	160k
2	1.00	1.00	1.00	1.00	1.00	1.00
3	0.75	0.76	0.73	0.68	0.64	0.62
4	0.63	0.63	0.60	0.57	0.58	0.63
5	0.40	0.46	0.48	0.48	0.51	0.53
6	0.03	0.04	0.06	0.09	0.15	0.16
7	0.02	0.03	0.04	0.06	0.09	0.16
8	0.01	0.02	0.02	0.04	0.06	0.11

Note. The compute metric is $advance_total + mpi_halo_exchange + mpi_migration + mpi_food_allreduce$, matching the hybrid sweep plots. Efficiency is computed with respect to the number of compute MPI processes only; rendering is excluded from the process count. For each combination of colony size and OpenMP threads per rank, the baseline is the smallest available MPI process count p_0. Each cell reports efficiency $E = S/(p/p_0)$.

TABLE IX
HYBRID MPI+OPENMP COMPUTE SPEEDUP FOR OMP=4 THREADS PER RANK.

Proc/Ants	5k	10k	20k	40k	80k	160k
2	1.00	1.00	1.00	1.00	1.00	1.00
3	1.18	1.15	1.12	1.13	1.16	1.14
4	0.53	0.67	0.78	1.05	0.92	1.29
5	0.06	0.09	0.13	0.21	0.30	0.59
6	0.04	0.06	0.09	0.15	0.26	0.45
7	0.03	0.05	0.07	0.11	0.19	0.34
8	0.03	0.04	0.05	0.09	0.15	0.27

Note. The compute metric is $advance_total + mpi_halo_exchange + mpi_migration + mpi_food_allreduce$, matching the hybrid sweep plots. Efficiency is computed with respect to the number of compute MPI processes only; rendering is excluded from the process count. For each combination of colony size and OpenMP threads per rank, the baseline is the smallest available MPI process count p_0. Each cell reports speedup $S = T(p_0)/T(p)$.

TABLE X
HYBRID MPI+OPENMP COMPUTE EFFICIENCY FOR OMP=4 THREADS PER RANK.

Proc/Ants	5k	10k	20k	40k	80k	160k
2	1.00	1.00	1.00	1.00	1.00	1.00
3	0.79	0.77	0.74	0.75	0.77	0.76
4	0.27	0.34	0.39	0.53	0.46	0.64
5	0.03	0.04	0.05	0.08	0.12	0.24
6	0.01	0.02	0.03	0.05	0.09	0.15
7	0.01	0.01	0.02	0.03	0.05	0.10
8	0.01	0.01	0.01	0.02	0.04	0.07

Note. The compute metric is $advance_total + mpi_halo_exchange + mpi_migration + mpi_food_allreduce$, matching the hybrid sweep plots. Efficiency is computed with respect to the number of compute MPI processes only; rendering is excluded from the process count. For each combination of colony size and OpenMP threads per rank, the baseline is the smallest available MPI process count p_0. Each cell reports efficiency $E = S/(p/p_0)$.

to cause significant increases in the time associated with migration, which means that although the computation time improves, the improvement is not sufficiently significant with respect to MPI parallelization when the total computation time is considered, as can be seen in the results for two threads per subdomain, which is the number of threads per subdomain that showed the best result in the test simulations.

It is also worth highlighting that when the number of MPI processes is increased and, in addition, the number of threads per rank is increased, each thread receives very few ants and the cost of coordinating those threads begins to consume an important part of the computation time, as can be seen in Fig. 5. For this reason, in the implementation of the hybrid model, too many threads per process become inefficient when the local subdomain is no longer sufficiently large, because the useful work per thread is not sufficiently significant and the cost of coordination and distribution of the results grows disproportionately with respect to the performance gain that it provides in the computation.

REFERENCES

[1] M. Dorigo, V. Maniezzo, and A. Colorni, "Ant system: Optimization by a colony of cooperating agents," *IEEE Transactions on Systems, Man, and Cybernetics—Part B: Cybernetics*, vol. 26, no. 1, pp. 29–41, Feb. 1996.